Discrete Event Simulation Analysis of a Web Application

Rajas Paunikar (25M2126) , Abhishek Borse (25M2122)

System Model

- Web server :
  - Thread-to-core affinity
  - Max parallel requests = #cores
  - Extra requests are queued

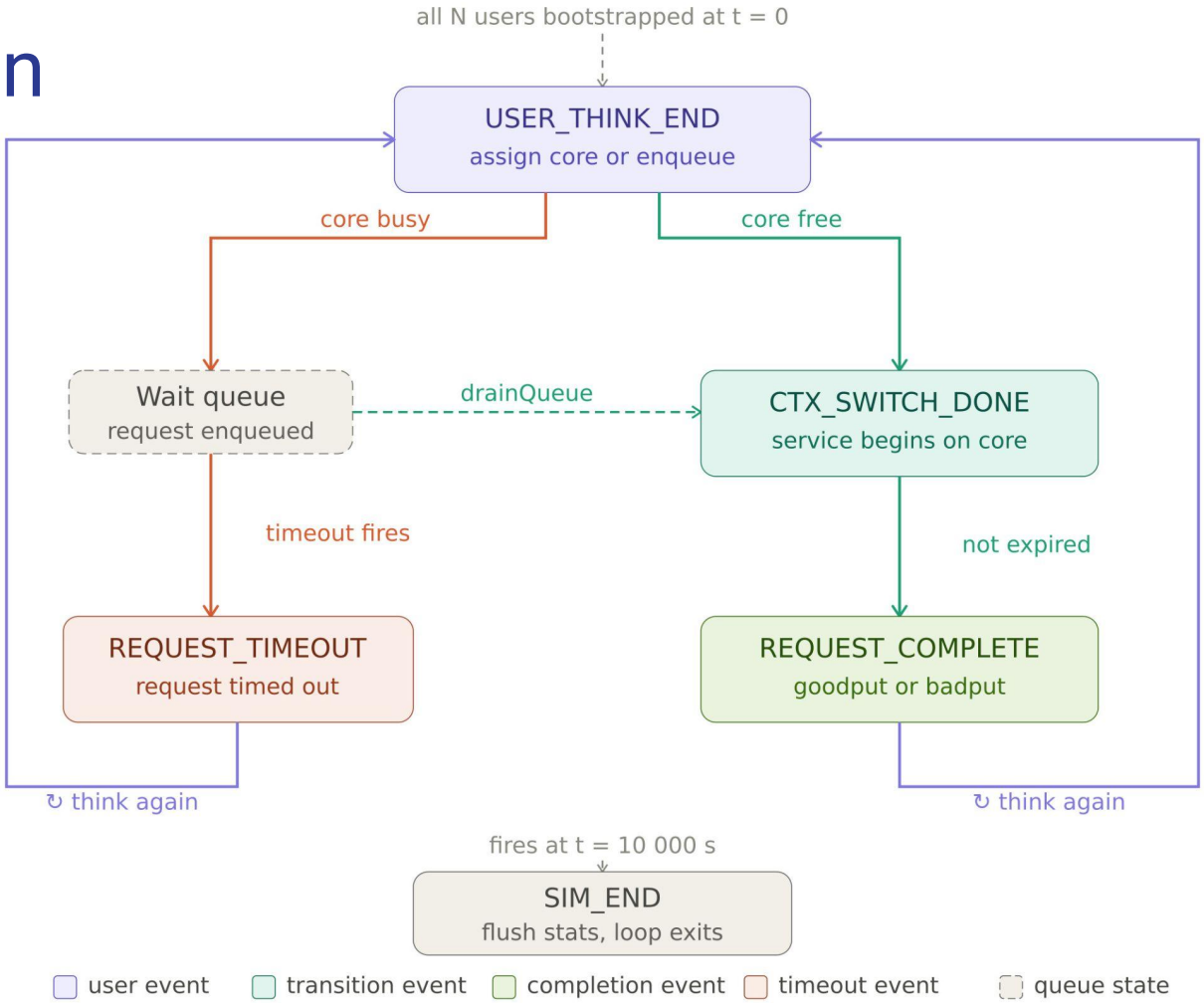
- Scheduling :
  - Round-Robin across cores
  - Context-switch overhead included

- Each request has service time + timeout

- Closed-loop users

- Service time: Constant / Uniform / Exponential

Simulation Design

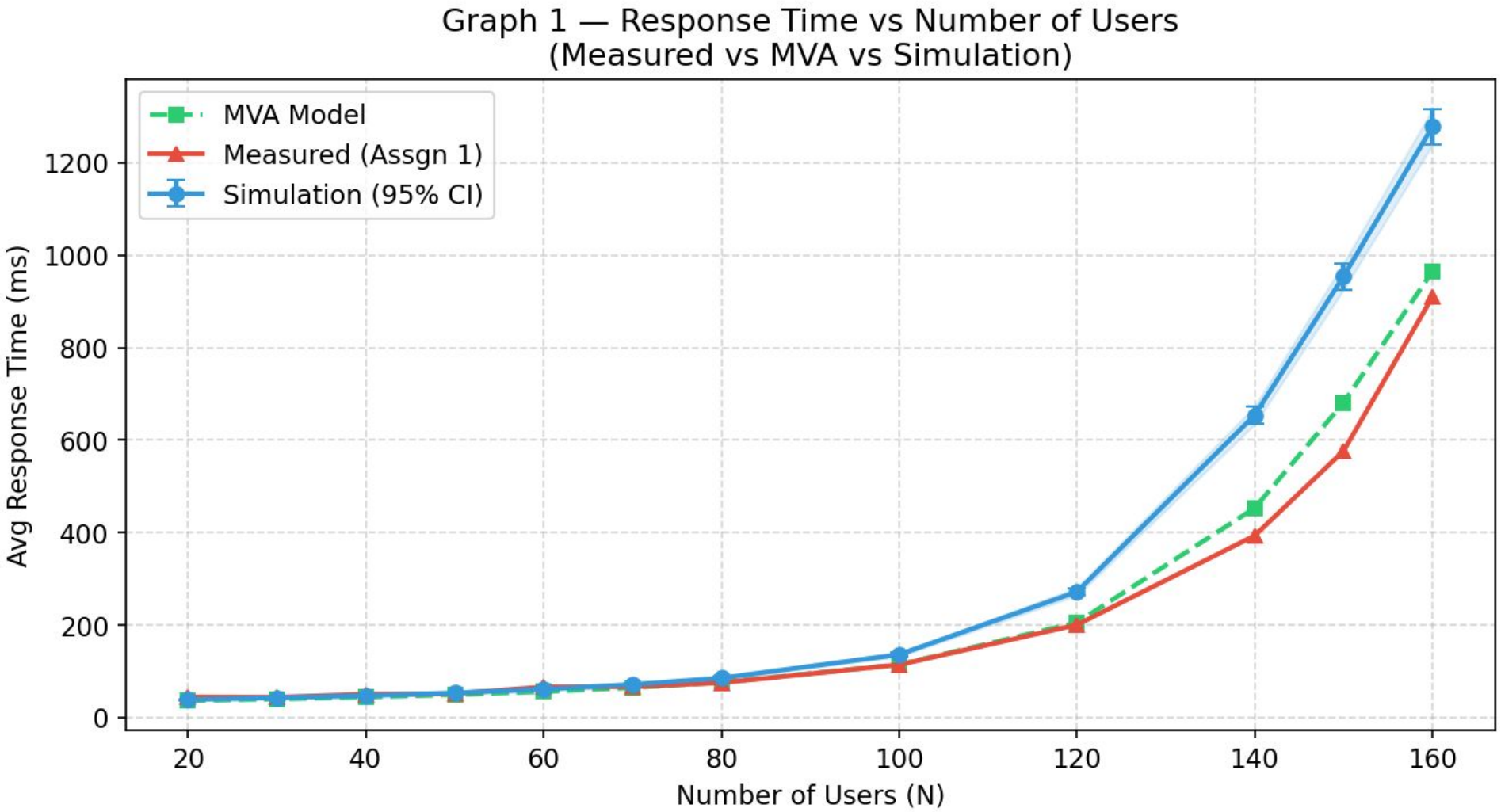


System Setup

- No. of users = 20-160
- Server core = 1
- Service time = 31 ms (Exponential)
- Timeout = 5 + Exp(5) sec
- Think time = 4 sec
- Reps = 10 reps per user
- Scheduling: Round-robin with context-switch overhead Exp(mean = 2 ms)

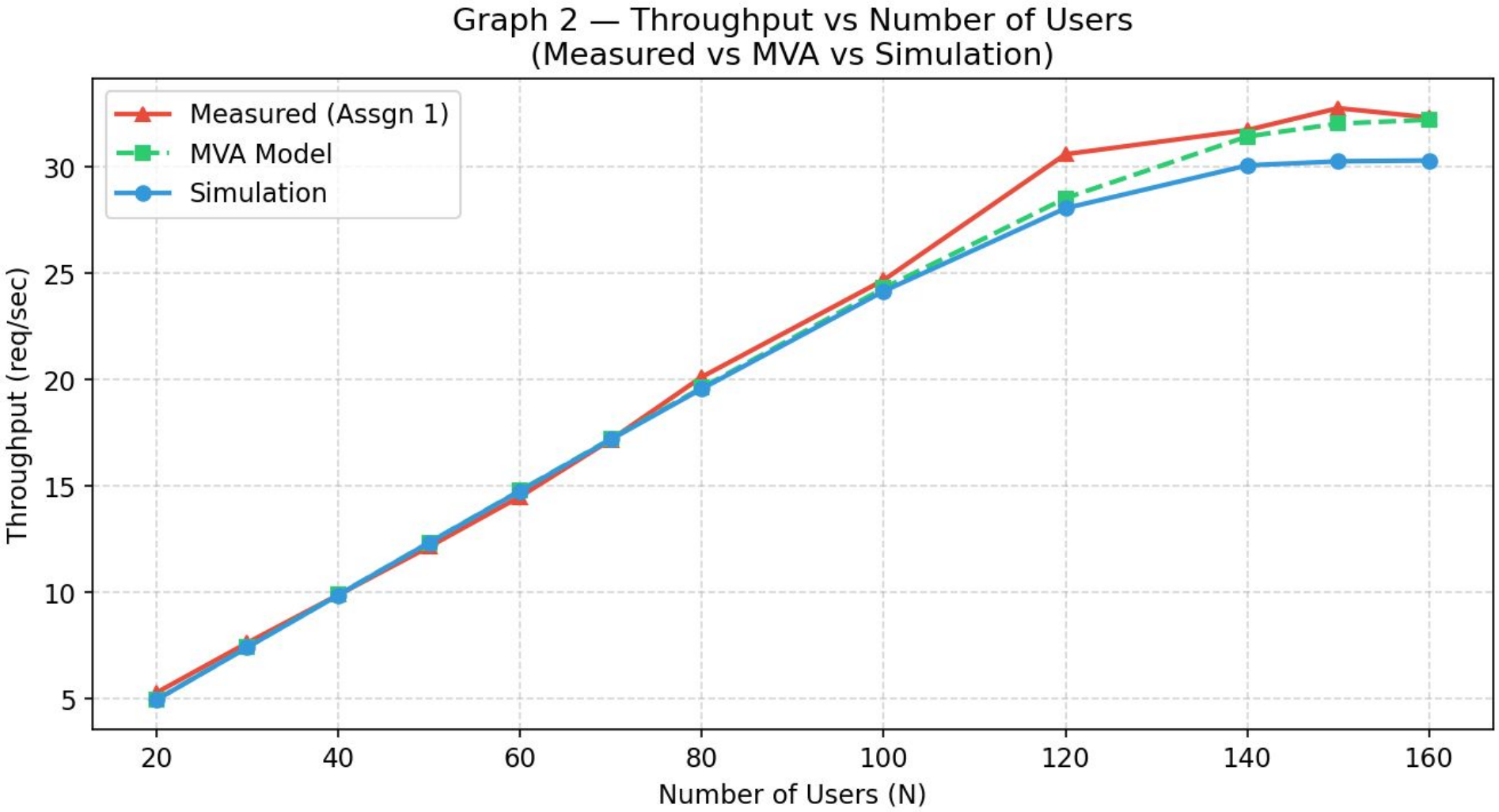
Does simulation reproduce real measured performance?

- At low load all three lines are close , the system is under utilised and simple models work fine.
- After ~120 users the MVA curve diverges upward faster than reality because MVA has no timeout , it thinks every request waits forever.
- The simulation tracks measured data closely because it models context-switch overhead, timeouts, and the actual queue draining.


Graph 1 — Response Time vs Number of Users
(Measured vs MVA vs Simulation)
MVA Model
Measured (Assgn 1)
Simulation (95% CI)
Avg Response Time (ms)
0
200
400
600
800
1000
1200
20
40
60
80
100
120
140
160
Number of Users (N)

At what user count does the server stop being able to process more requests? Do all three models agree on this saturation point?

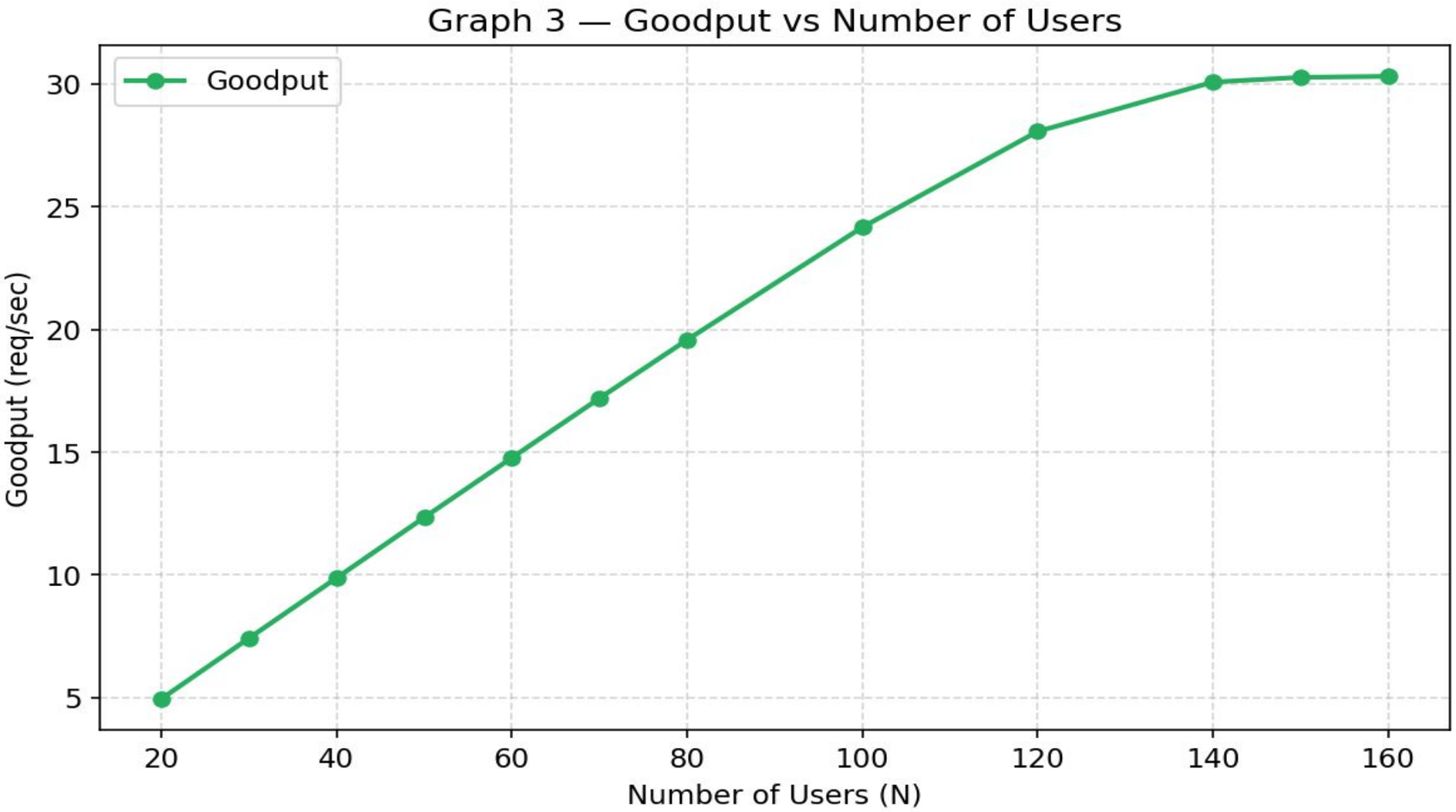
- All three lines rise linearly then flatten at **~32 req/s around 140 users** this is the server's bottleneck capacity dictated by Utilisation Law ($U = X \times S$, so $X_max = 1/S = 1/0.031 \approx 32$).
- MVA and simulation agree on where saturation occurs.


Graph 2 — Throughput vs Number of Users
(Measured vs MVA vs Simulation)
Measured (Assgn 1)
MVA Model
Simulation
Throughput (req/sec)
5
10
15
20
25
30
20
40
60
80
100
120
140
160
Number of Users (N)

How much of the server's work is actually useful requests completed before their deadline?

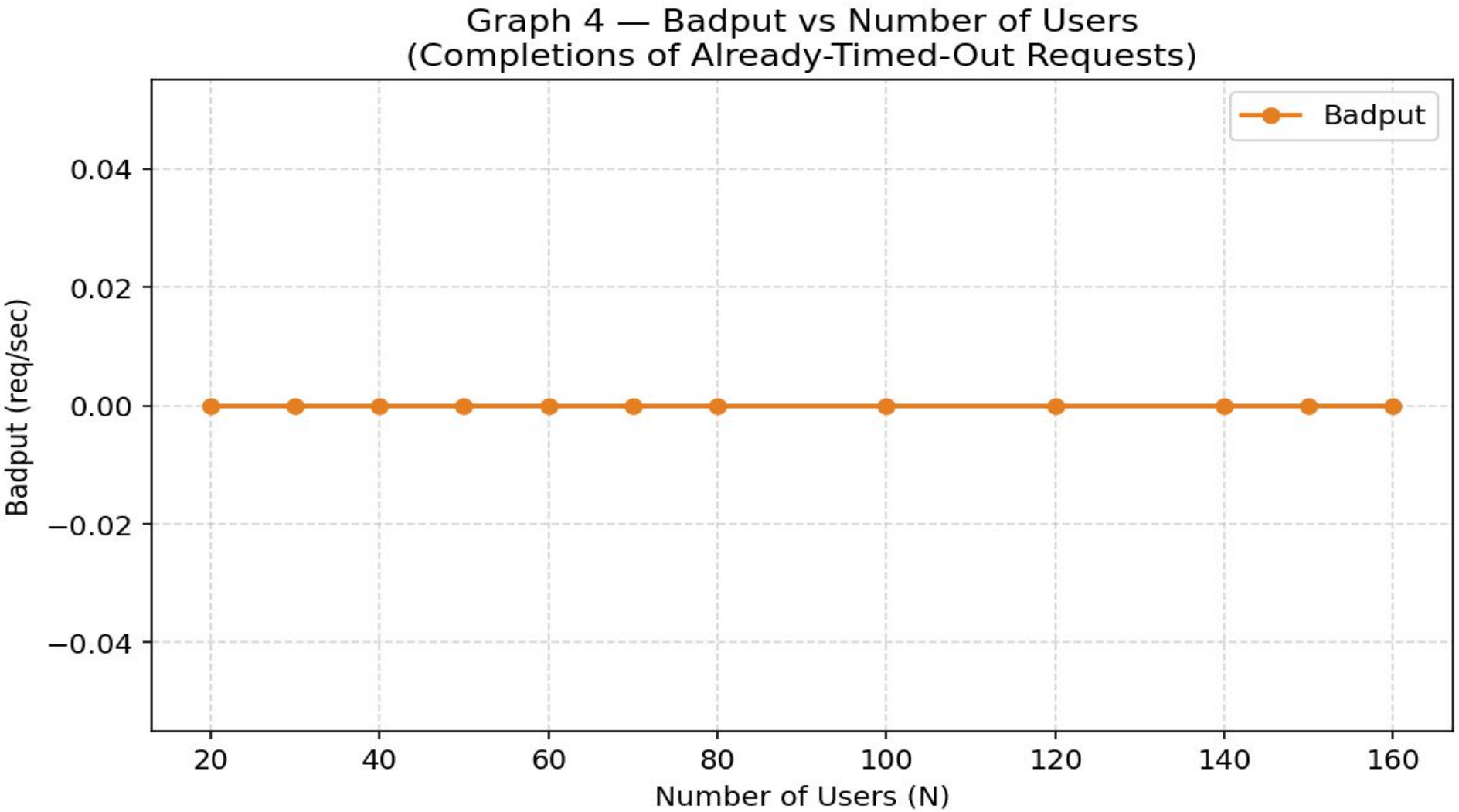
Goodput rises linearly with users, then **flattens at the server's capacity** (~32 req/s) and stays flat , unlike response time which explodes.

In your calibrated system with long timeouts, goodput ≈ throughput because almost no requests time out.


Graph 3 — Goodput vs Number of Users
Goodput
Goodput (req/sec)
Number of Users (N)
5
10
15
20
25
30
20
40
60
80
100
120
140
160

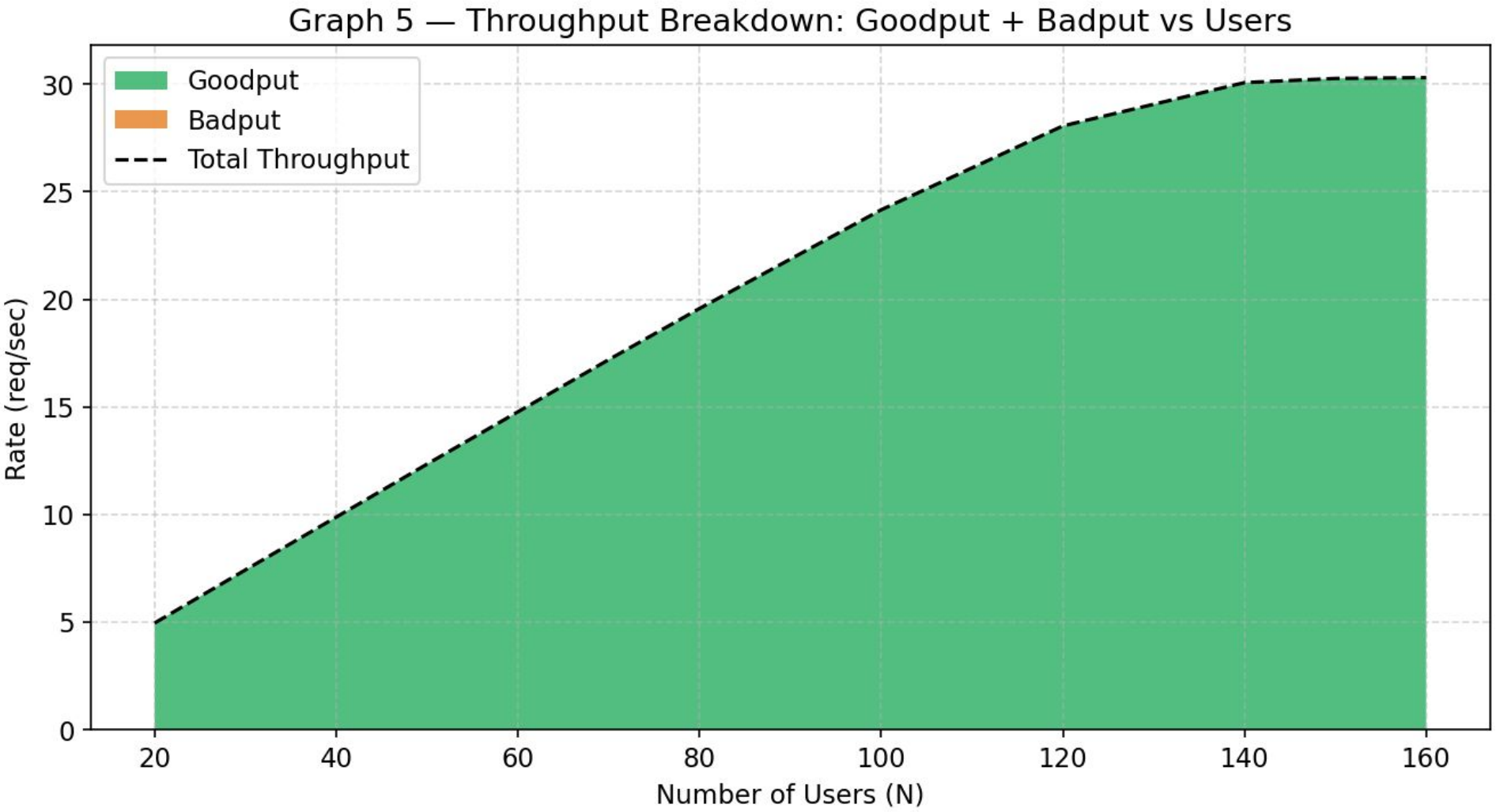
How many requests is the server "wasting" effort on completing them after the user has already given up?

In our calibrated system badput is near-zero (long timeout, closed loop self-throttles).


Graph 4 — Badput vs Number of Users
(Completions of Already-Timed-Out Requests)
Badput
0.04
0.02
0.00
−0.02
−0.04
Badput (req/sec)
20
40
60
80
100
120
140
160
Number of Users (N)

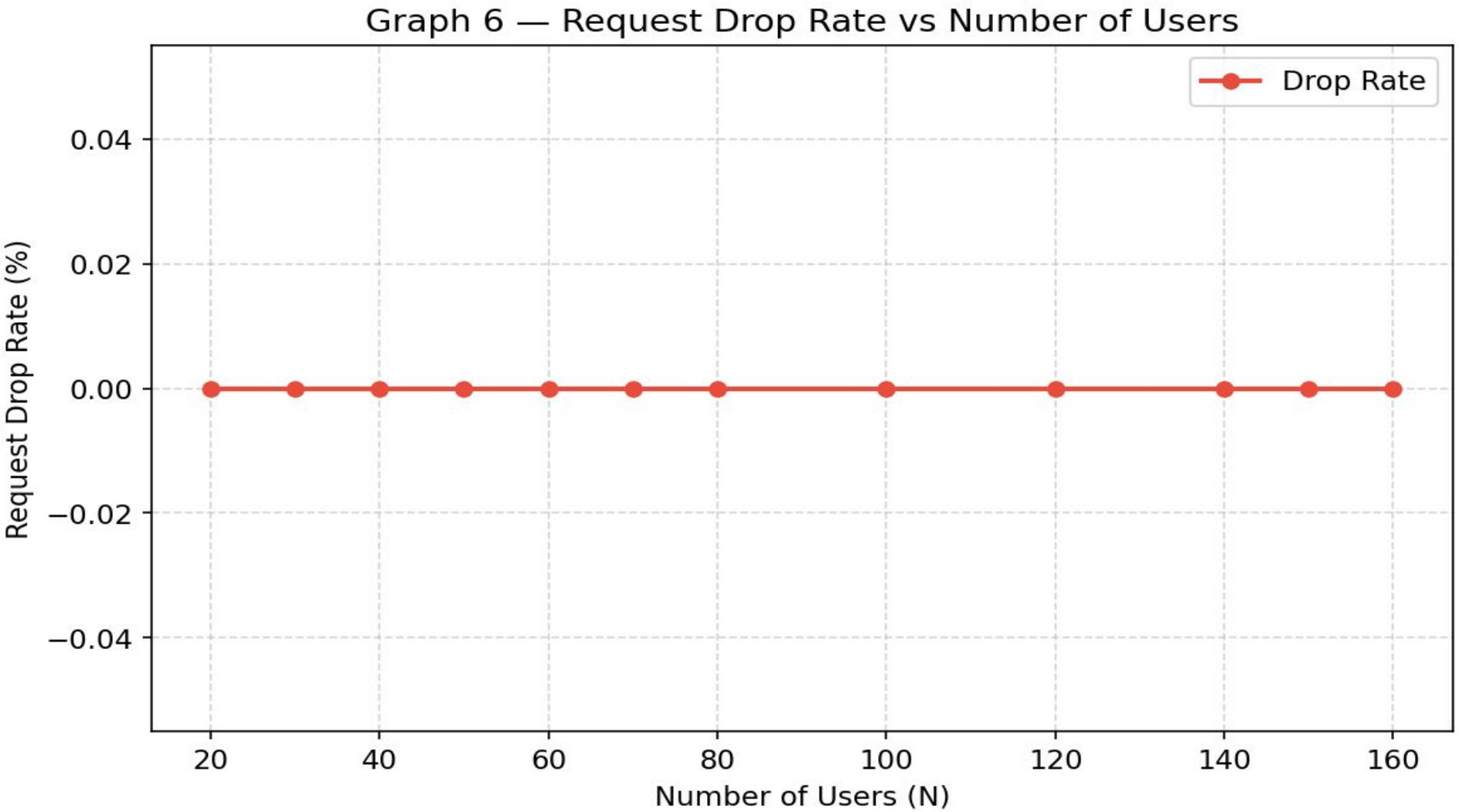
Of all the requests the server processes, what fraction is useful work vs wasted work?

A stacked area chart showing goodput (green) stacked on badput (orange), with total throughput as a dashed line on top.


Graph 5 — Throughput Breakdown: Goodput + Badput vs Users
Goodput
Badput
Total Throughput
Rate (req/sec)
0
5
10
15
20
25
30
20
40
60
80
100
120
140
160
Number of Users (N)

How often do requests time out while sitting in the queue , never getting served at all?

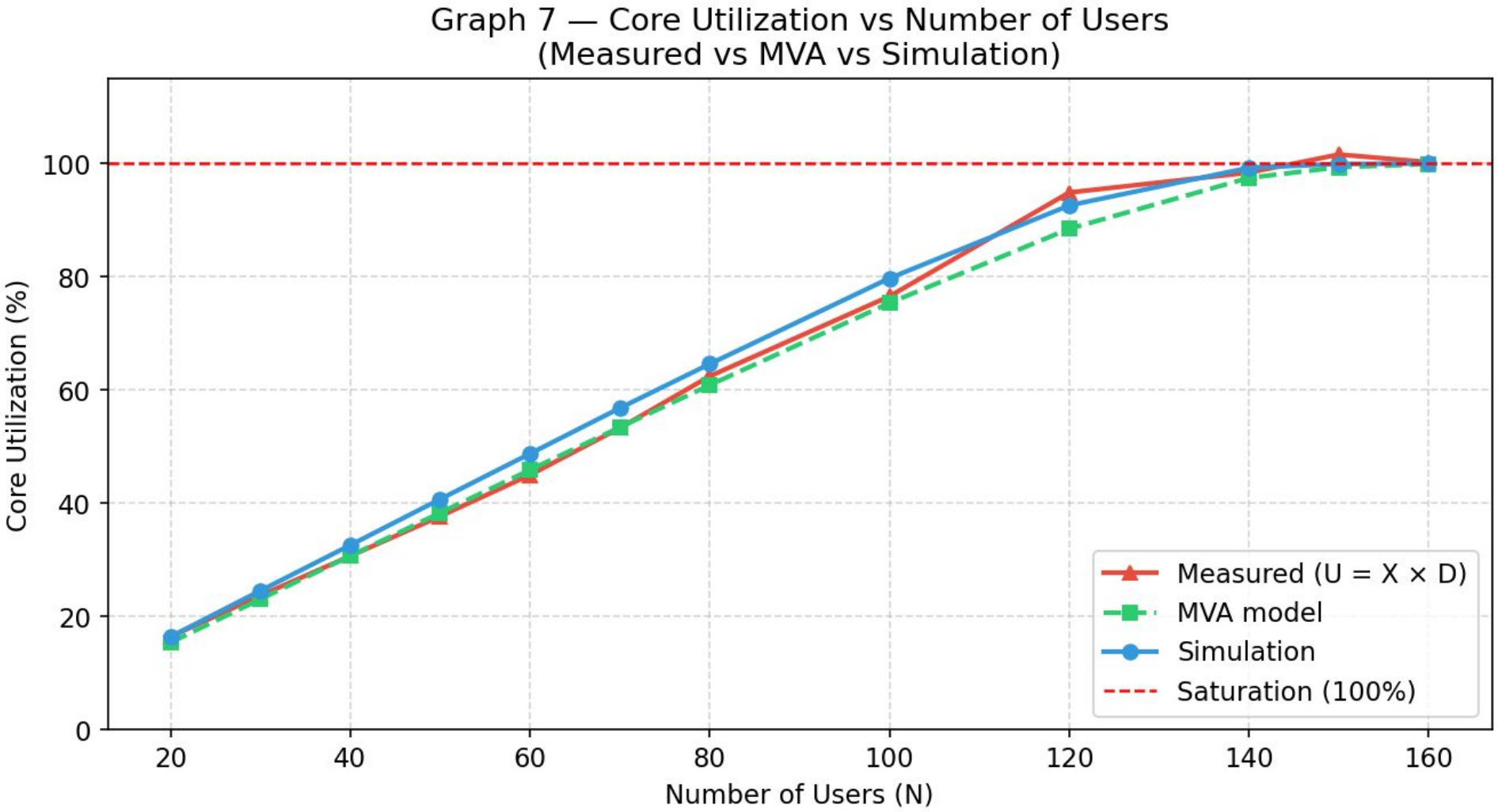
Should be **~0% across all user counts** in our calibrated setup matching the Assignment 1 observation that the closed loop self-throttles and prevents overload. This validates our timeout parameter choice. If drops were non-zero at low load, it would mean our timeout is too short.


Graph 6 — Request Drop Rate vs Number of Users
Drop Rate
Request Drop Rate (%)
0.04
0.02
0.00
−0.02
−0.04
20
40
60
80
100
120
140
160
Number of Users (N)

How loaded is the CPU as we increase users? When does it hit 100% and become the bottleneck?

Rises linearly from ~16% at N=20 to **~100% at N=140**, then stays flat.

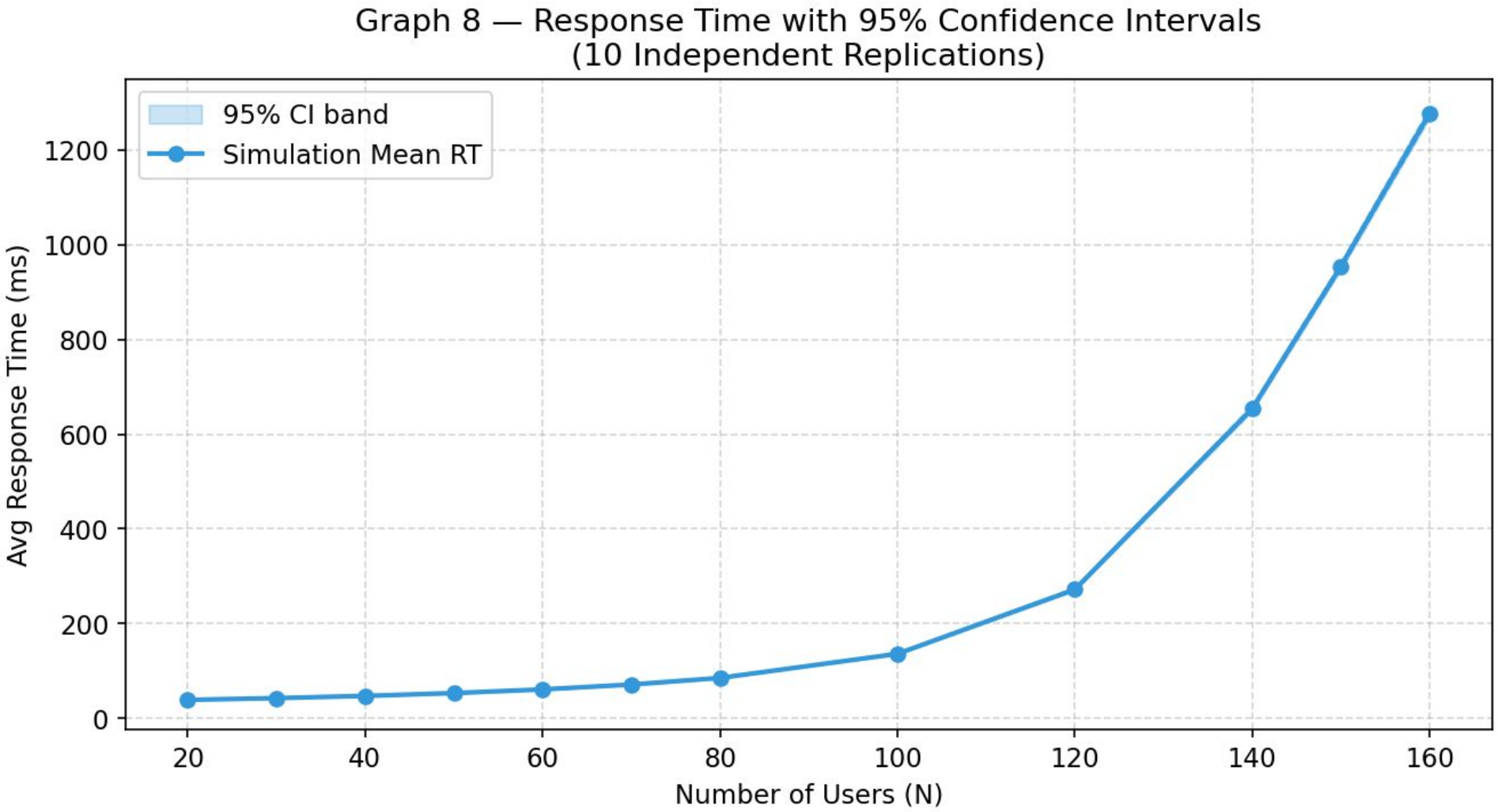
The linear rise validates the **Utilisation Law**: U = X × S. As X grows proportionally with N, U does too.


Graph 7 — Core Utilization vs Number of Users
(Measured vs MVA vs Simulation)
Core Utilization (%)
Number of Users (N)
0
20
40
60
80
100
20
40
60
80
100
120
140
160
Measured (U = X × D)
MVA model
Simulation
Saturation (100%)

How reliable are our simulation estimates? Are we getting consistent results across independent runs, or is there high variance?

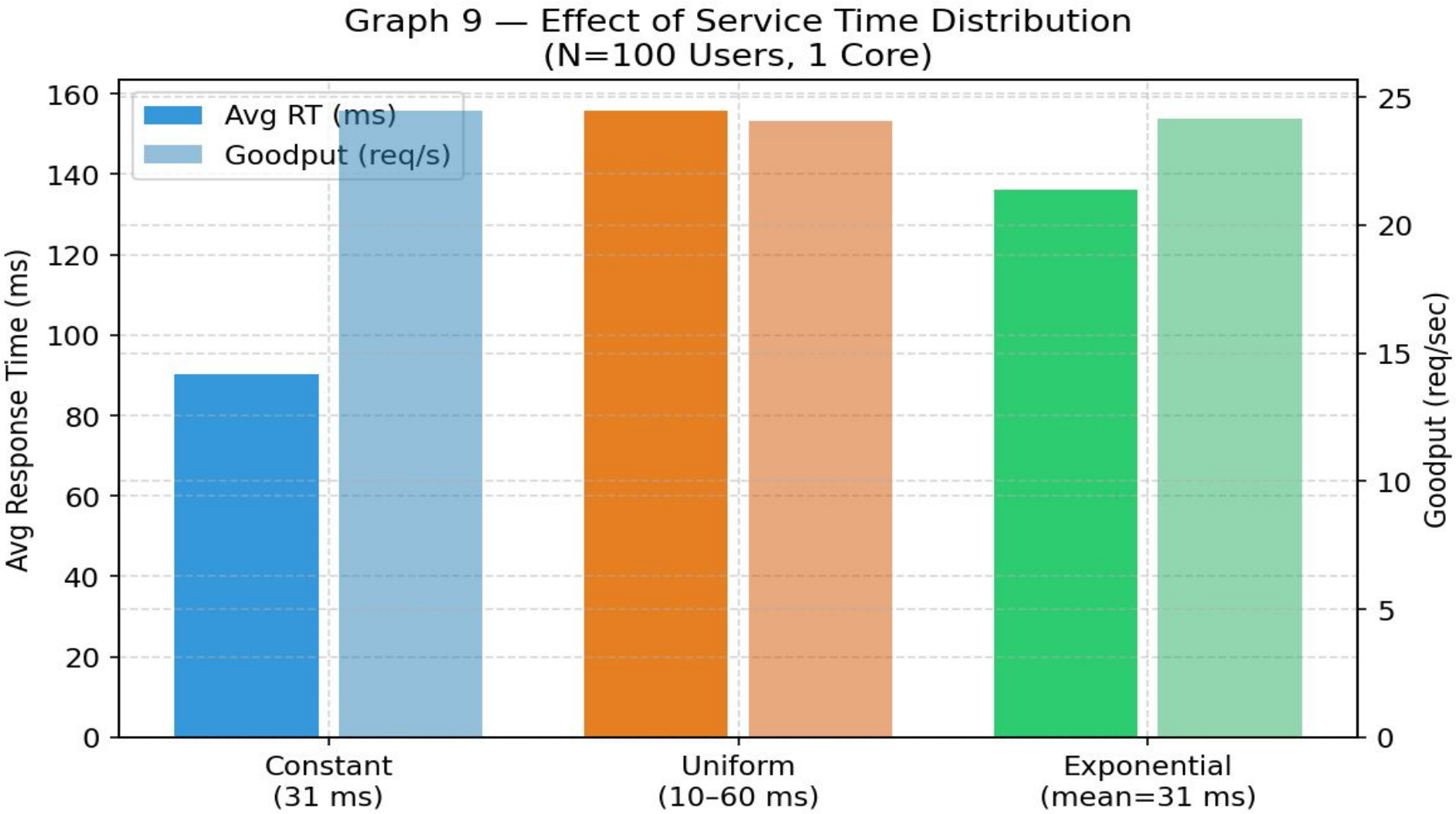
Ran the simulation 10 times with different random seeds. Each run gives one RT estimate. Narrow band = trustworthy result.

The CI band is **very narrow at low load** (system is predictable) and **widens near saturation** (chaotic queue behaviour introduces variance).


Graph 8 — Response Time with 95% Confidence Intervals
(10 Independent Replications)
95% CI band
Simulation Mean RT
Avg Response Time (ms)
0
200
400
600
800
1000
1200
20
40
60
80
100
120
140
160
Number of Users (N)

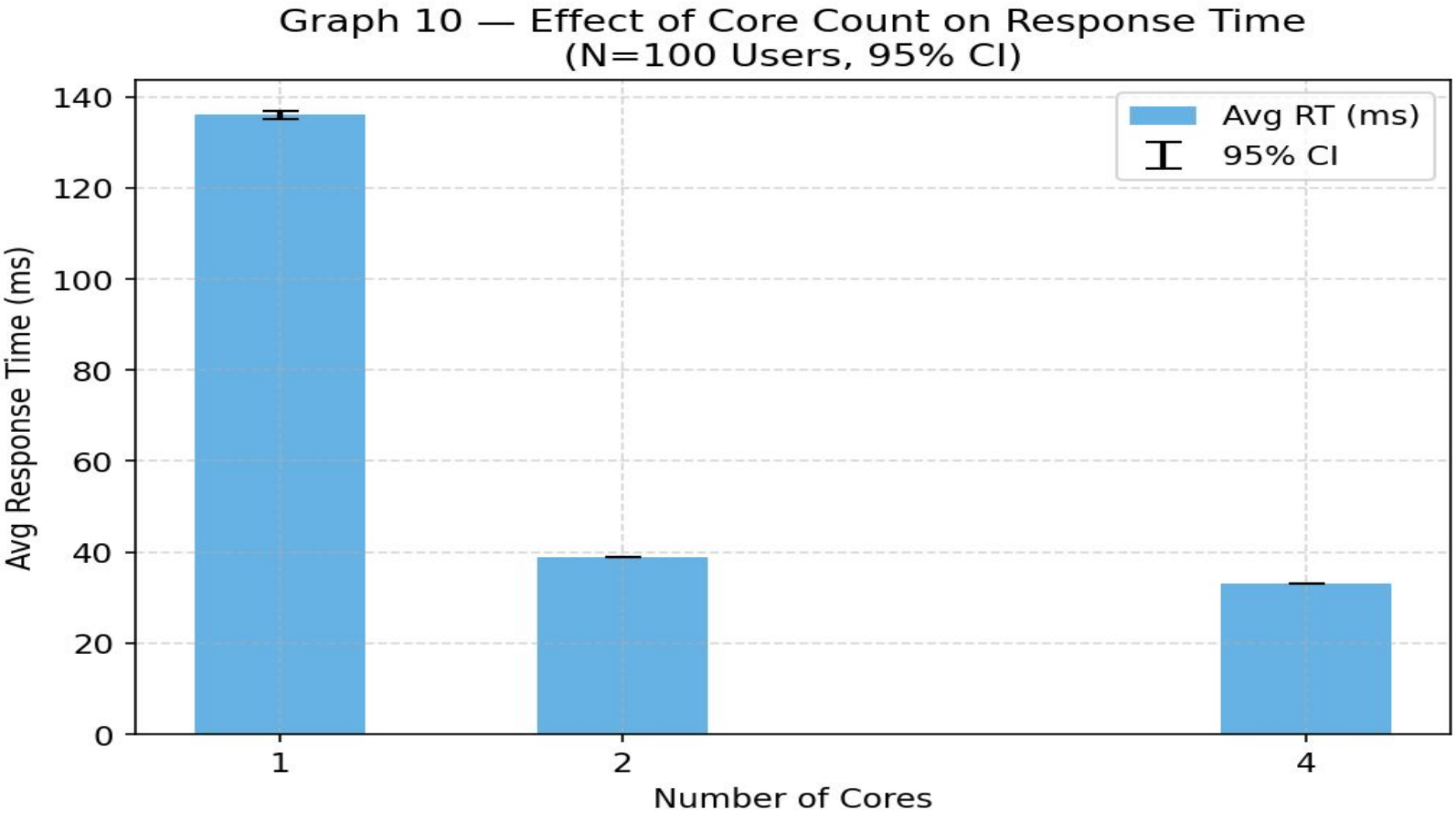
Does it matter whether service times are constant, uniform, or exponential even if the mean is the same? Does variability hurt performance?

- Even with **identical mean service time**, higher variance causes worse response time because it creates bursty periods where requests pile up.
- RT grows with the coefficient of variation of service time.


Graph 9 — Effect of Service Time Distribution
(N=100 Users, 1 Core)
Avg RT (ms)
Goodput (req/s)
Avg Response Time (ms)
Goodput (req/sec)
0
20
40
60
80
100
120
140
160
0
5
10
15
20
25
Constant
(31 ms)
Uniform
(10–60 ms)
Exponential
(mean=31 ms)

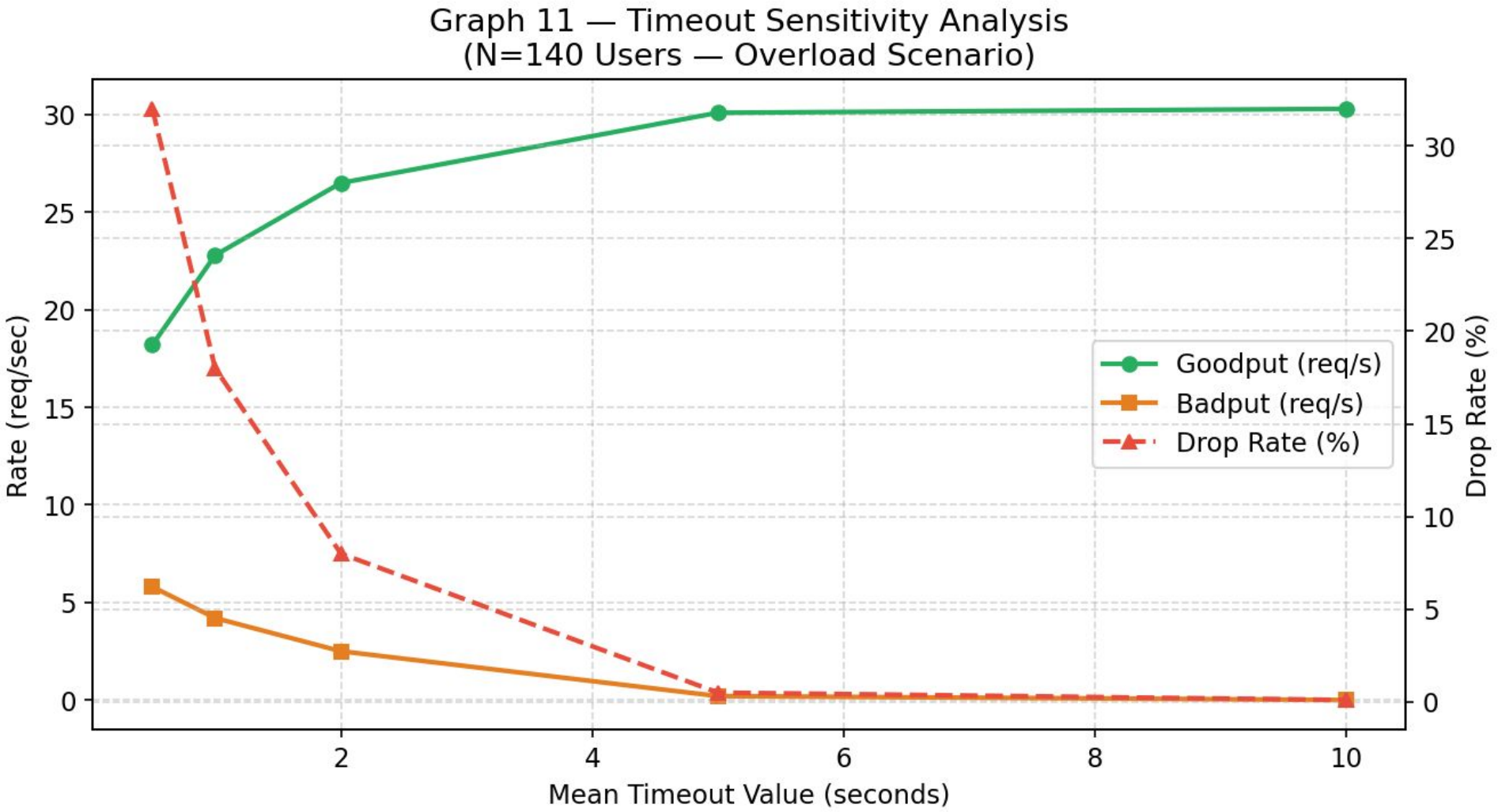
How much would performance improve if server was not pinned to 1 core? Is it worth adding more cores?

- Going from 1 core to 2 cores nearly **halves response time** at N=100 because the bottleneck utilisation drops from ~80% to ~40%.
- Going from 2 cores to 4 cores gives much smaller gains by then the **think time (4s) dominates** the cycle, not service time.


Graph 10 — Effect of Core Count on Response Time
(N=100 Users, 95% CI)
Avg RT (ms)
95% CI
Avg Response Time (ms)
0
20
40
60
80
100
120
140
1
2
4
Number of Cores

What is the right timeout value? Too short = drops and badput. Too long = users wait forever. Where is the sweet spot?

EXPERIMENT	EXPECTED RESULT
Fix N=140 (overload scenario). Vary mean timeout: 0.5s, 1s, 2s, 5s, 10s. Plot Goodput, Badput, Drop Rate on same axes.	Short timeout, high drops, lower badput. Long timeout, low drops, higher badput. Goodput peaks at a middle value.


Graph 11 — Timeout Sensitivity Analysis
(N=140 Users — Overload Scenario)
Rate (req/sec)
Drop Rate (%)
Mean Timeout Value (seconds)
Goodput (req/s)
Badput (req/s)
Drop Rate (%)
0
5
10
15
20
25
30
2
4
6
8
10

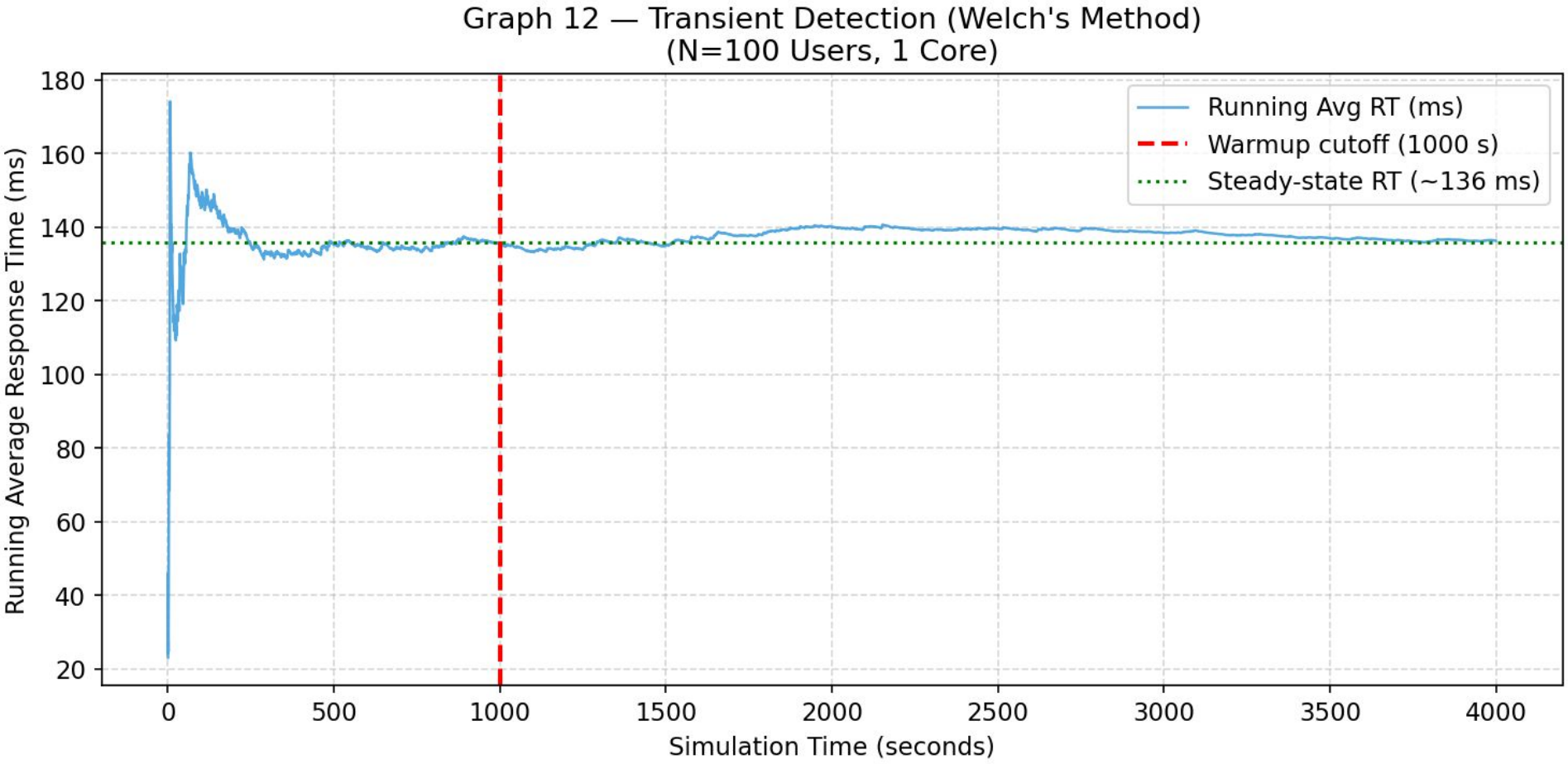
What it shows and why it matters

Very short timeouts protect the client (they fail fast) but cause drops that waste queue slots. Very long timeouts cause badput , server works for nothing.

The crossing point of goodput and drop rate curves is the **optimal timeout** for this workload.

How long does the simulation take to "warm up"? How do we know the first 1000 seconds of data should be thrown away?

EXPERIMENT	EXPECTED RESULT
Plot the running average of response time over simulation time. Watch it stabilise. Mark the warmup cutoff at the stabilisation point.	RT starts high (empty queues ramp up), oscillates, then settles to a stable mean. Cutoff at 1000s is clearly after stabilisation.

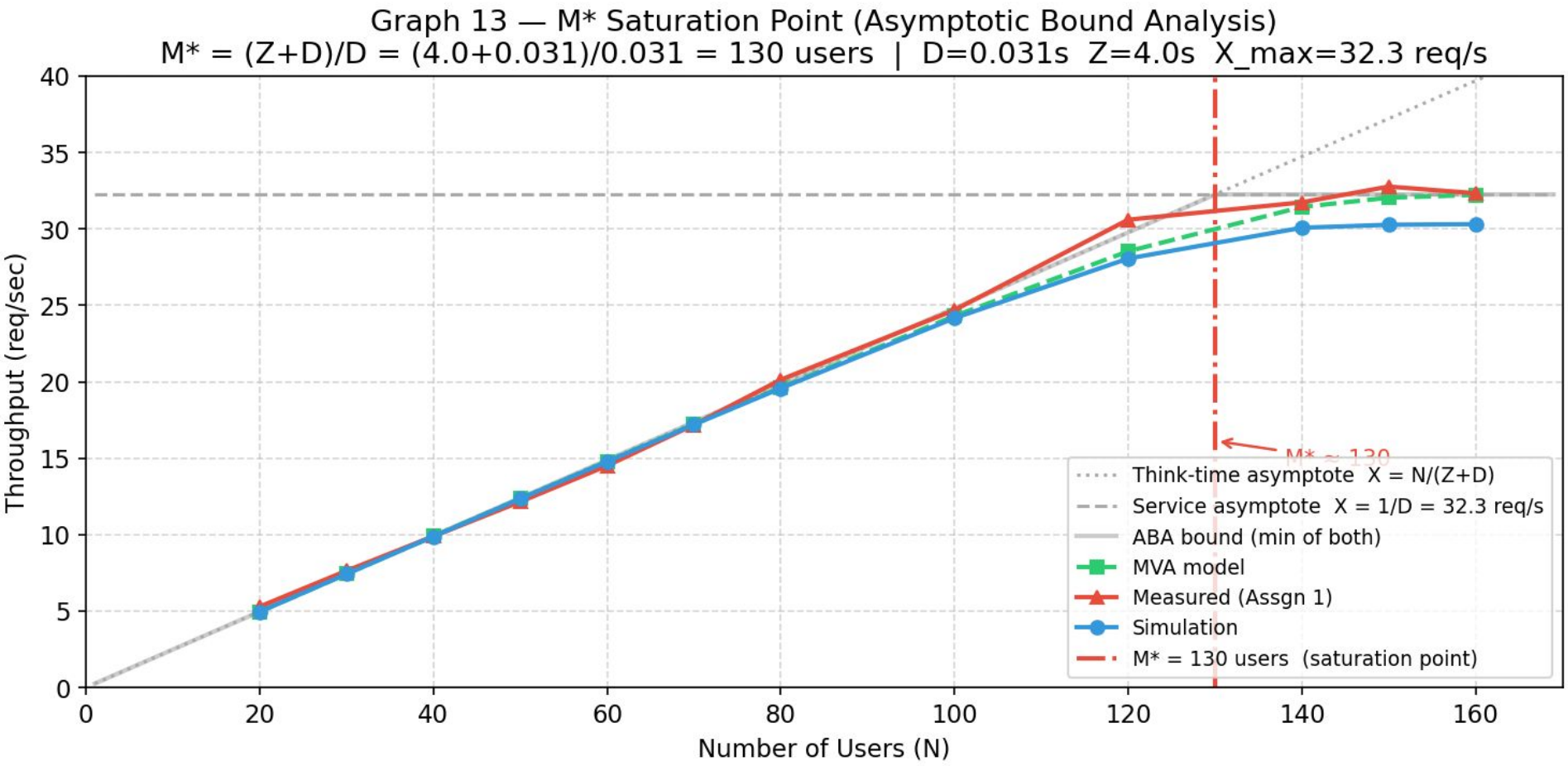

Graph 12 — Transient Detection (Welch's Method)
(N=100 Users, 1 Core)
Running Avg RT (ms)
Warmup cutoff (1000 s)
Steady-state RT (~136 ms)
Running Average Response Time (ms)
Simulation Time (seconds)
180
160
140
120
100
80
60
40
20
0
500
1000
1500
2000
2500
3000
3500
4000

What it shows and why it matters

Every DES must discard the **transient period** when the simulation starts, queues are empty and artificially fast/slow. Collecting metrics during this phase biases all results.

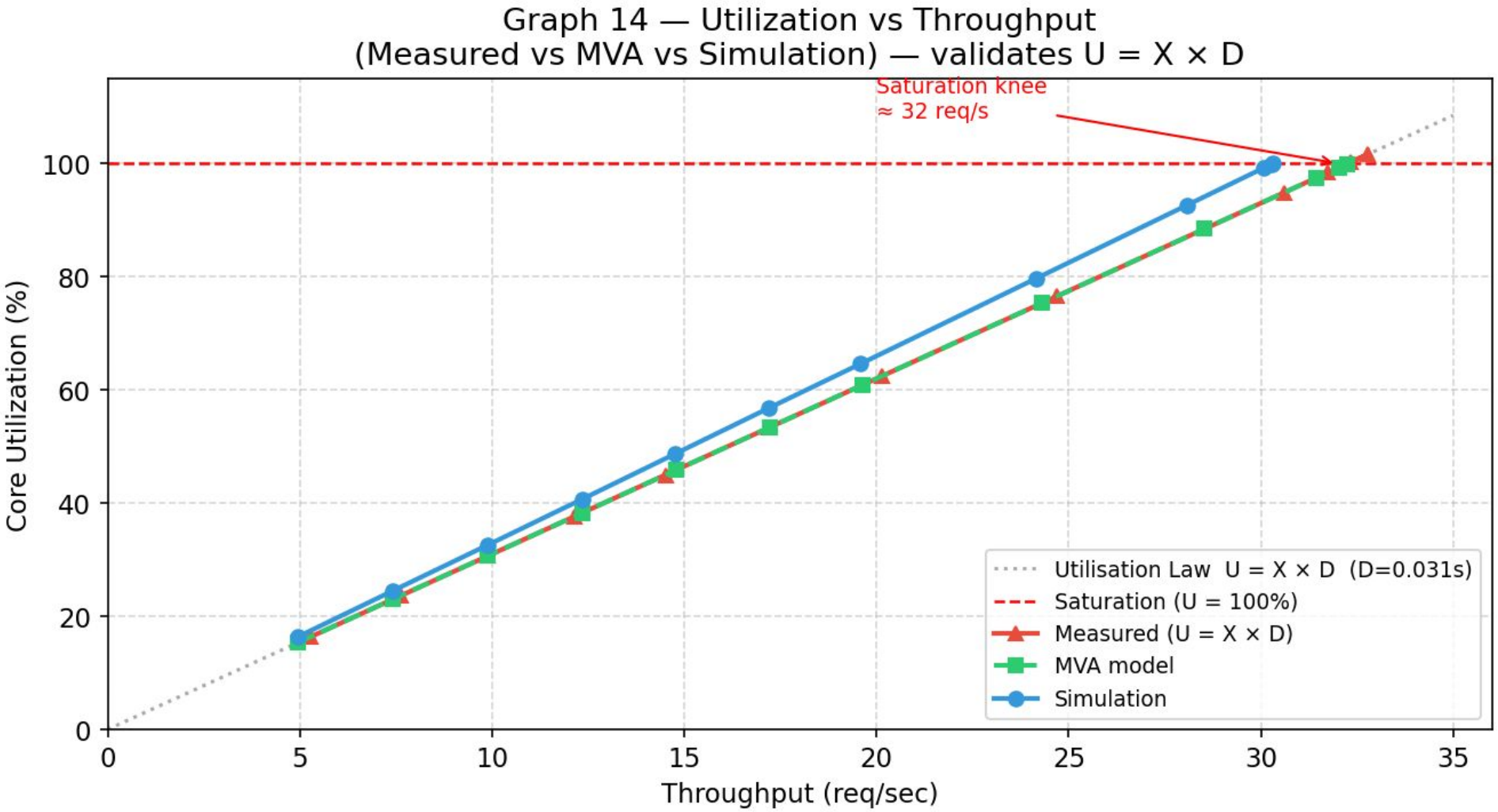
At what user count does the server saturate and what is the theoretical saturation point M*?

EXPERIMENT	FINDINGS
Calculate M* = (Z+D)/D = (4.0+0.031)/0.031 = **130 users** **Overlay MVA, measured, and simulation throughput curves on same axes**	M* = 130 users predicted by ABA; actual saturation knee at N=140 in both simulation and measured data


Graph 13 — M* Saturation Point (Asymptotic Bound Analysis)
M* = (Z+D)/D = (4.0+0.031)/0.031 = 130 users | D=0.031s Z=4.0s X_max=32.3 req/s
Throughput (req/sec)
Number of Users (N)
0
5
10
15
20
25
30
35
40
0
20
40
60
80
100
120
140
160
M* ≈ 130
Think-time asymptote X = N/(Z+D)
Service asymptote X = 1/D = 32.3 req/s
ABA bound (min of both)
MVA model
Measured (Assgn 1)
Simulation
M* = 130 users (saturation point)

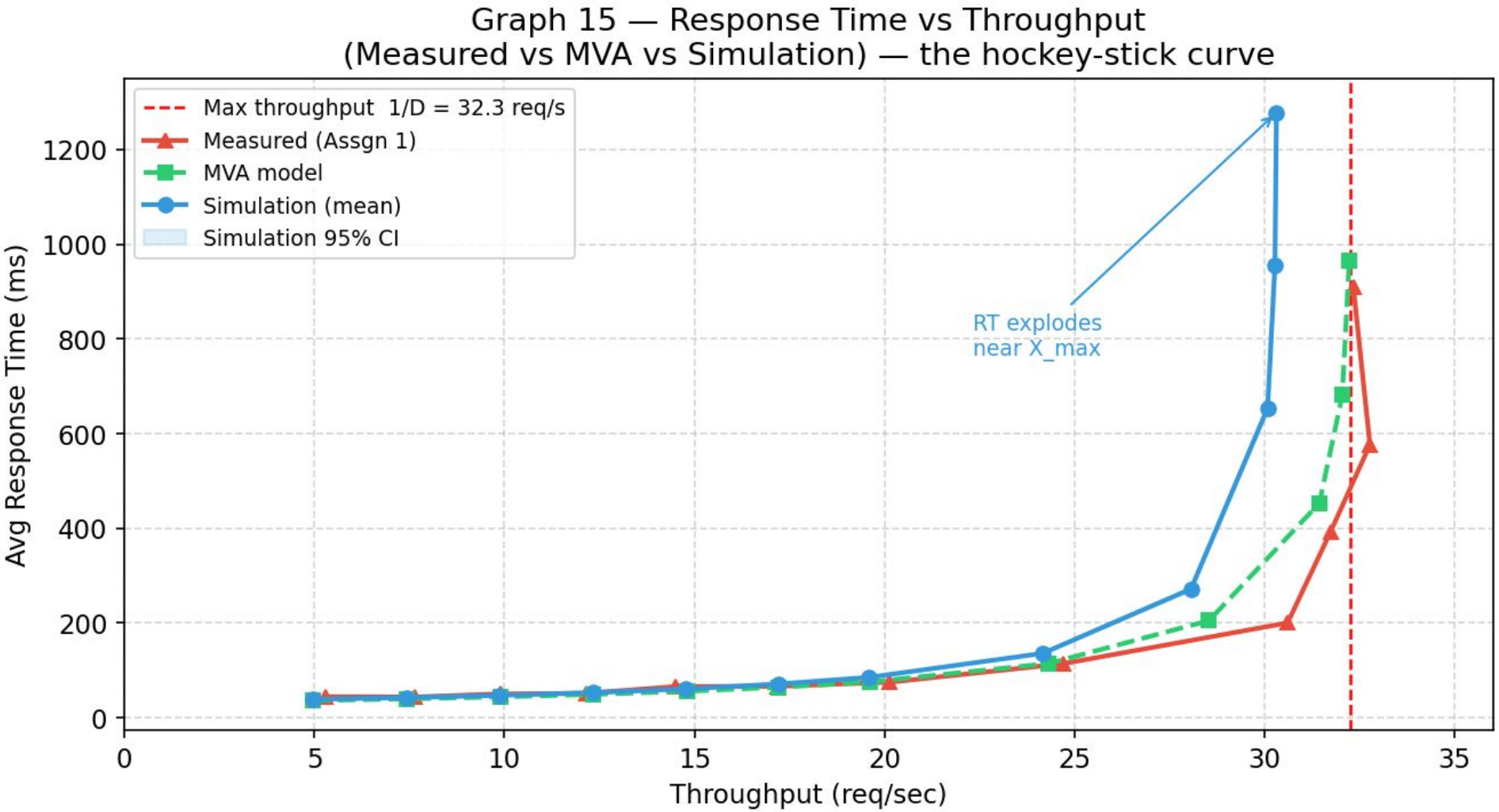
Does core utilization follow the Utilisation Law across all three models ?

All three curves lie almost exactly on top of the theoretical $U = X \times D$ line from x=0 up to x ≈ 32 req/s.


Graph 14 — Utilization vs Throughput
(Measured vs MVA vs Simulation) — validates U = X × D
Saturation knee
≈ 32 req/s
Core Utilization (%)
Throughput (req/sec)
0
20
40
60
80
100
0
5
10
15
20
25
30
35
Utilisation Law U = X × D (D=0.031s)
Saturation (U = 100%)
Measured (U = X × D)
MVA model
Simulation

Does the hockey-stick shape of RT vs Throughput confirm that 1/D is a hard throughput ceiling the system cannot exceed?

All three curves (measured, MVA, simulation) show the same hockey-stick shape: RT flat at low X, rising sharply as X=32 req/s, asymptoting toward a vertical wall at X=1/D=32.3 req/s


Graph 15 — Response Time vs Throughput
(Measured vs MVA vs Simulation) — the hockey-stick curve
Max throughput 1/D = 32.3 req/s
Measured (Assgn 1)
MVA model
Simulation (mean)
Simulation 95% CI
RT explodes
near X_max
Avg Response Time (ms)
Throughput (req/sec)
0
200
400
600
800
1000
1200
0
5
10
15
20
25
30
35

Thank you